



**UNIVERSITÀ
DI TORINO**

Corso di Laurea Triennale in Informatica

Progetto di Sistemi Operativi

Simulazione di un Ufficio Postale

Turno 2

Componenti Gruppo

Angela Luca, 1100008

Benslimane Bilal, 1094638

Chirio Tommaso, 1099300

A.A 2024-2025

Indice

1. Introduzione	2
2. Configurazioni	3
3. Processi	4
a. Direttore	4
b. Operatore	5
c. Utente	5
d. Erogatore Ticket	5
4. Esempio di esecuzione	6
5. Risorse	7
6. Considerazioni finali	7

1. Introduzione

Il progetto consiste nel simulare il funzionamento di un ufficio postale tramite l'utilizzo delle risorse IPC e dell'uso dei seguenti processi/risorse.

- **direttore** → è il processo principale (Ufficio Posta) che gestisce l'intera simulazione (genera sportelli, operatori e gli utenti). Si occupa di:
 - fa partire la sessione eseguendo i giorni lavorativi e i vari processi figli;
 - stampa a video e nel file .csv le statistiche ottenute, giornaliera e dell'intera simulazione;
 - alla fine, rilascia tutte le risorse IPC che vengono utilizzate (shared memory, code di messaggi).
- **erogatoreTicket** → processo che si occupa di erogare il ticket in base al servizio richiesto secondo la tabella. Il tempo indicato è utilizzato per generare un tempo casuale intorno al 50%.

SERVIZIO	TEMPARIO (IN MINUTI)
INVIO E RITIRO PACCHI	10
INVIO LETTERE E RACCOMANDATE	8
PRELIEVI E VERSAMENTI BANCOPOSTA	6
PAGAMENTO BOLLETTINI POSTALI	8
ACQUISTO PRODOTTI FINANZIARI	20
ACQUISTO OROLOGI E BRACCIALETTI	20

- **sportello** → risorse che offrono un servizio specifico, scelto in maniera casuale, e che cambiano durante le giornate della simulazione
- **operatore** → processi in quantità *NOF_WORKERS* che hanno un orario di lavoro e decidono durante la giornata attiva di andare o meno in pausa
 - vengono eseguiti all'inizio della giornata occupando uno sportello in base al proprio lavoro;
 - decidono in maniera arbitraria se andare in pausa oppure no attraverso una probabilità *P_PAUSE*
 - Servono l'utente eseguendo una nanosleep in base al tempario e al servizio richiesto
- **utente** → processi in quantità *NOF_USERS* che vengono serviti in base al ticket/servizio richiesto. Decidono secondo *P_SERV_MAX* e *P_SERV_MIN* se andare o meno in posta.

2. Configurazioni

Nel progetto vengono utilizzati due tipi di file, reperibili nella directory `data`:

1. **`config_timeout.conf`** → contiene valori utilizzati per terminare la simulazione correttamente
2. **`config_explode.conf`** → contiene valori che interrompono la simulazione in maniera prematura, ovvero esplode se il numero degli utenti in attesa a fine giornata è maggiore di `EXPLODE_THRESHOLD`

Queste configurazioni devono essere copiate e incollate all'interno di `post_office.conf`, file di configurazione principale che viene utilizzato dal direttore per salvarsi i seguenti valori:

- **`NOF_WORKER_SEATS`** → numero di postazioni disponibili
- **`NOF_WORKERS`** → numero totale di lavoratori
- **`NOF_USERS`** → numero totale di utenti
- **`SIM_DURATION`** → durata complessiva della simulazione, ovvero i giorni lavorativi
- **`N_NANO_SECS`** → tempo a disposizione per la simulazione in nanosecondi
- **`P_SERV_MIN`** → percentuale minima assegnata ad un servizio
- **`P_SERV_MAX`** → percentuale massima assegnata ad un servizio
- **`P_PAUSE`** → percentuale di un operatore che va in pausa
- **`NOF_PAUSE`** → numero di pause concesse a ciascun lavoratore
- **`EXPLODE_THRESHOLD`** → soglia minima di utenti in coda per attivare l'esplosione della simulazione

Nella directory `include` invece abbiamo creato due file `.h` (`config.h` e `functions.h`) che contengono rispettivamente:

1. `config.h`

- **`MAX_SPORTELLI`** → numero massimo di sportelli disponibili nel sistema
- **`MAX_LAVORATORI`** → numero massimo di lavoratori disponibili nel sistema
- **`NUM_SERVIZI`** → numero totale dei servizi offerti
- **`tempario[NUM_SERVIZI]`** → tempi di erogazione dei servizi
- **`Config`** → struttura in cui vengono salvati tutti i valori di `post_office.conf`, inclusi altri valori:
 - **`DAYS_LEFT`** → giorni rimanenti alla fine della simulazione
 - **`sportelli[MAX_SPORTELLI]`** → configurazione degli sportelli per tipo di servizio.
 - **`lavoratori[MAX_LAVORATORI]`** → mappatura dei lavoratori e dei loro servizi.

- **StatsDay/StatsSim** → strutture che contengono rispettivamente le statistiche della giornata e dell'intera simulazione
- **struct msgbuf** → struttura per i messaggi nella coda tra operatori e sportelli
- **Sportello** → struttura che contiene il tipo di lavoro assegnato
- **WorkerMsg** → struttura usata dai lavoratori per scambiare messaggi con gli utenti
- **TicketMsg** → struttura usata da utenti per comunicare con erogatoreTicket
 - *mtype*: tipo del messaggio (1 per richiesta, PID per risposta)
 - *servizio*
 - *ticket*: numero del ticket (inviato solo in risposta)
 - *pid* (solo in richiesta)
- **Ticket/TicketPtr** → struttura che rappresenta il ticket
 - *type*: servizio richiesto
 - *time*: tempo necessario per erogare il servizio

2. **functions.h** → comprende tutte le funzioni utilizzate all'interno del codice dei vari processi

3. Processi

a. Direttore

Il processo direttore è il nucleo della intera simulazione, poiché è il responsabile della creazione dei processi, coordina l'inizio e la fine delle giornate tramite i semafori e i segnali e si occupa anche della stampa delle statistiche in output, sia a video che nel file in formato csv.

Al suo avvio, crea tutti gli operatori e gli utenti in base alle configurazioni disponibili nella `shared_memory` e assegna agli sportelli un servizio in maniera casuale per ogni giorno. Quest'ultimo viene fatto tramite una coda di messaggi, che contiene il tipo di lavoro assegnato, ovvero un valore da 1 a 6.

A fine di ogni giornata, lancia dei segnali di kill con messaggio di SIGUSR1 (ultimo giorno) e SIGUSR2 (altri giorni) per ogni processo utente e operatore per evitare che rimangano processi in attesa.

Alla fine della simulazione riporta, oltre alle statistiche ([statsSim.csv](#)), se il programma è terminato correttamente oppure c'è stata un `explode`, causata dalla coda degli utenti in attesa che ha superato il limite `EXPLODE_THRESHOLD`.

b. Operatore

L'operatore si occupa di erogare il servizio in base al tempario e alla richiesta dell'utente.

Ogni giorno viene assegnato ad uno sportello con il suo stesso lavoro, e se ci sono più operatori rispetto agli sportelli disponibili vengono messi in una coda di messaggi, per evitare situazioni di busy waiting. Alla fine del suo servizio rilascia lo sportello, che verrà occupato successivamente da un altro operatore.

Comunicano con gli utenti per vedere il tipo di servizio richiesto e di conseguenza servirli tramite la WorkerMsg.

c. Utente

L'utente, all'inizio di ogni giornata, decide con una probabilità data se andare o meno in posta. Se ha esito positivo, prende un ticket tramite TicketMsg e verifica se esiste uno sportello disponibile.

Si reca alla posta dopo aver scelto un orario e si inserisce in coda a uno degli sportelli che offre il servizio da lui richiesto. Il servizio viene fatto tramite una comunicazione diretta tra l'utente e l'operatore con WorkerMsg.

d. Erogatore Ticket

Eroga il ticket su richiesta dell'utente tramite TicketPtr, dove l'utente invia tramite TicketMsg il servizio richiesto e il suo PID su mtype. Al suo interno inserisce il tipo di servizio richiesto e il tempo per erogarlo.

L'erogatore ticket lo invia all'utente tramite la coda TicketMsg, dove in risposta invia il PID in mtype e in pid inserisce la richiesta, incrementando il numero di ticket con un contatore.

L'utente infine utilizzerà quel ticket per essere servito in uno sportello.

4. Esempio di esecuzione

```
bilal@HPBilal:~/Documents/Sistemi Operativi/SO/Progetto/bin$ ./direttore
[DIRECTOR] Sportello 0 che offre il servizio: 1
[DIRECTOR] Sportello 1 che offre il servizio: 4
[DIRECTOR] Sportello 2 che offre il servizio: 2
[DIRECTOR] Erogatore ticket con pid:32321
[TICKET] Inizializzazione Erogatore Terminata
[WORKER] 0 inizializzazione terminata
[WORKER] 1 inizializzazione terminata
[WORKER] 3 inizializzazione terminata
[WORKER] 2 inizializzazione terminata
[WORKER] 4 inizializzazione terminata
[WORKER] 5 inizializzazione terminata
[USER] 32328 inizializzazione terminata
[DIRECTOR] Inizializzazione USER Terminata
[USER] 32329 inizializzazione terminata
[DIRECTOR] Inizializzazione EROGATORE Terminata
[DIRECTOR] Inizializzazione LAVORATORI Terminata
SESSIONE INIZIATA
```

```
[WORKER] Operatore Sleeping till day starts
[WORKER] Operatore Sleeping till day starts
[WORKER] Lavoratore è uscito dal semaforo.
[WORKER] Lavoratore è uscito dal semaforo.
[WORKER] Lavoratore 4, PID: 32326, tipoLavoro=6
[WORKER] Lavoratore 5, PID: 32327, tipoLavoro=6
[USER] 32328 ha ricevuto il ticket 1
[USER] 32328: Pagamento bollettini postali (4)
[USER] Immissione in coda da parte di 32328...
[USER] Invio il messaggio con pid: 32328
[USER] Fine invio messaggio con pid: 32328
[WORKER] Lavoratore 1 svolge il lavoro di 32328...
[USER] ha inviato la richiesta!
[USER] 32328 ha finito, torna a casa.
SPORTELLLO TIPO 1 RIMOSSO
[USER] Sleeping till day starts
[USER] Sleeping till day starts
[DIRECTOR] Giorni rimanenti: 0
[DIRECTOR] Fine giornata.

[DIRECTOR] Giorni finiti, chiudo tutto
```

5. Risorse

- **Semafori** → usati per sincronizzare i vari processi in queste situazioni:
 - Inizio e fine delle giornate
 - Inizializzazione dei processi
- **Shared Memory** → usate per condividere dati tra i processi per le seguenti situazioni:
 - Statistiche giornaliere e dell'intera simulazione
 - Configurazioni di partenza della simulazione, lette dal file `post_office.conf`
- **Sportelli** → risorsa generata giornalmente dal direttore con tipo `Lavoro casuale`. Vengono usati dagli operatori per poter lavorare
- **Code di messaggi** → usate per le comunicazioni tra due processi:
 - `WorkerMsg` tra utenti e operatori;
 - `TicketMsg` tra utenti ed erogatore `Ticket`;
 - `struct msgbuf` tra operatori e direttore (`sportelli`);
- **execl** → esegue un processo specificando il percorso e l'eseguibile, con gli argomenti e il `NULL` alla fine.

6. Considerazioni finali

Durante la fase di progettazione, abbiamo riscontrato alcune difficoltà legate principalmente alla gestione della sincronizzazione tra processi e all'utilizzo delle risorse IPC, in particolare della memoria condivisa.

Per quanto riguarda la shared memory principale (`Config`), si è deciso di utilizzare un array statico a dimensione fissa. Questa scelta è stata dettata dalla necessità di garantire la corretta condivisione dei dati tra i processi, obiettivo che sarebbe risultato più complesso da raggiungere con strutture dinamiche.

Alla fine, il programma raggiunge la terminazione coprendo tutti i casi limite e gestendo correttamente i casi di timeout oppure di `explode`.